

2026-06-10-SP7-testing-qa-plan

SP7 — Testing & QA (cross-cutting) — Implementation Plan

Revision	1
Created	2026-06-10
Last modified	2026-06-10
Status	draft
Authority	master roadmap docs/superpowers/specs/2026-06-10-llms-access-master-roadmap.md §SP7 + analysis docs/superpowers/specs/analysis/2026-06-10-F-parallel-dynamic-testing.md §3
Scope	helix_code/ (inner Go app dev.helix.code) + submodules/helix_qa + submodules/challenges, cross-cutting over SP1/SP2/SP4/SP5/SP6
Evidence rule	every claim cites a real file:line/dir/. ABSENT marked explicitly. No fabrication. PLANNING only — nothing executed.

Constraints binding every task below: anti-bluff §11.4 family (captured evidence on every PASS; metadata-only/config-only/absence-of-error/grep-without-runtime PASS forbidden); §11.4.98 fully-automated (no human-in-loop during test exec); §11.4.123 rock-solid-proof-or-deep-research; §11.4.116 sync-channel; §11.4.125/§11.4.134 code-review gate iterate-until-GO; §11.4.135 regression-guard per fixed defect; §11.4.85 stress+chaos floors; §1.1 paired-mutation per harness/gate; §11.4.71 + §11.4.113 fetch→investigate→merge-onto-latest-main→fast-forward push (no force). Each task = **RED** → **impl** → **GREEN+evidence** → **rollback**.

Table of contents

1. [Verified ground truth](#)
2. [Test-type coverage matrix \(15 types\)](#)
3. [Phase A — close ddos/scaling/ux/thin-ui local gaps](#)
4. [Phase B — HelixQA fetch/pull-to-latest + pointer bump](#)
5. [Phase C — autonomous-QA-session wiring \(§11.4.98 + §11.4.116\)](#)
6. [Phase D — independent code-review-agent gate per closure](#)
7. [Phase E — regression-guard registration per fixed defect \(incl. D-1..D-7\)](#)
8. [Operator decisions required](#)
9. [Summary](#)

1. Verified ground truth

Confirmed by direct inspection this session:

- **Strong HelixCode-local coverage:** unit (helix_code/tests/unit/, Makefile test:107), integration (helix_code/tests/integration/, test-integration-full:222, -tags=integration), e2e (helix_code/tests/e2e/ + tests/e2e/challenges/executor.go, test-e2e-full:228), security (helix_code/tests/security/{owasp_test.go, authentication_test.go, authorization_test.go, tools_security_scanners Makefile security-scan*:751-775), chaos+stress (helix_code/tests/stresschaos/{chaos.go, stresschaos.go} + ~30 per-pkg *_stress_test.go/*_chaos_test.go, Makefile stress-chaos:114), performance/benchmarking (helix_code/tests/performance/{benchmark_test.go, competitor_baseline_test.go, pgo_test.go, pprof_harness_test-benchmark:157, pgo-refresh:90).
 - **Anti-bluff meta-tests EXIST and are real:** helix_code/tests/stresschaos/stresschaos_meta_test.go:80-193 plants deadlock/leak/error-rate/below-floor/chaos-panic and asserts detection; Makefile stress-chaos-meta:121. This is the canonical §1.1 paired-mutation pattern every new harness in this plan MUST replicate.
 - **Regression dir EXISTS:** helix_code/tests/regression/{critical_paths_test.go, server_timeout_test.go} — but is NOT yet a standing per-defect §11.4.135 guard registry (no RED_MODE polarity switch, no D-N guard rows).
 - **HelixQA is STALE:** submodules/helix_qa at pin v4.0.0-393-g4d2dcb2; README banner submodules/helix_qa/README.md:3 and ledger submodules/helix_qa/docs/test-coverage.md both say **round 219, 2026-05-19** → 22 days stale vs 2026-06-10. Ledger declares all 15 slots FILLED (rows 1-15, test-coverage.md:50-66).
 - **ddos/scaling/ux/ui assets live ONLY in HelixQA**, NOT in helix_code/tests/: submodules/helix_qa/challenges/scripts/{ddos_health_flood_challenge.sh, scaling_horizontal_challenge.sh, ux_end_bank_banks/ddos-ratelimit-comprehensive.yaml.find helix_code/tests -iname '*ddos*' -o -iname '*scaling*' -o -iname '*ux*' → **empty**. This is the coverage-bluff risk per CONST-050(B): a delegated slot is acceptable ONLY if the HelixQA asset truly exercises HelixCode with captured evidence.
 - **Bridge EXISTS:** helix_code/internal/helixqa/{wrapper.go, translator.go}; Makefile helixqa-build:694, helixqa-test:707, helixqa-challenge:710, helixqa-bump-submodules:720; helix_code/go.mod requires digital.vasic.helixqa (replace → ../submodules/helix_qa).
 - **Real defects (roadmap ledger lines 43-49):** D-1 hardcoded 3-model list helix_agent/internal/handlers/completion.go:406 (SP2); D-2 CLI lists failed/pending as available helix_code/.../cmd/cli/main.go:1361 (SP1); D-3 dead secrets.LoadAPIKeys helix_code/internal/secrets/loader.go:30 (SP1); D-4 working-model filter loaded-never-applied adapter.go:175 (SP1); D-5 hardcoded provider lists openai_provider.go:203, anthropic_provider.go:205,... (SP1); D-6 stub CLI-agent returns hardcoded strings helix_agent/internal/clis/agents/qwencode/qwencode.go:101/115/137 (SP4); D-7 helix_code/go.mod lacks replace dev.helix.agent (SP4/SP5).
-

2. Test-type coverage matrix (15 types)

Columns: **exists-where today** (file:line or ABSENT) · **gap** · **owning-SP feature the type must cover**. SP map: SP1 model-access, SP2 HelixAgent-exposure, SP4 CLI-bridge, SP5 parallel/subagent, SP6 dynamic-flows.

#	Test type	Exists-where today (helix_code-local + delegated)
1	unit	helix_code/internal/**/*_test.go, tests/unit/; Makefile test:
2	integration	helix_code/tests/integration/ (api_integration_test.go, app integration-full:222 -tags=integration
3	e2e	helix_code/tests/e2e/ (complete_workflow_test.go, challenge comprehensive_matrix_test.go); test-e2e-full:228
4	full-automation	delegated → HelixQA cmd/helixqa autonomous + helix_code/tests/{comprehensive_test_suite.sh, hardware_test.go}; test-comp
5	security	helix_code/tests/security/{owasp, authn, authz, tools}_test scan*:751-775; test-security-full:234
6	ddos	ABSENT local → only HelixQA challenges/scripts/ddos_health_flow banks/ddos-ratelimit-comprehensive.yaml

#	Test type	Exists-where today (helix_code-local + delegated)
7	scaling	ABSENT local → only HelixQA challenges/scripts/scaling_horizon
8	chaos	helix_code/tests/stresschaos/chaos.go + per-pkg *_chaos_test chaos:114
9	stress	helix_code/tests/stresschaos/stresschaos.go + per-pkg *_stre worker/worker_pool_stress_test.go, task/, agent/, workflow/)
10	performance	helix_code/tests/performance/ {benchmark_test.go, competitor_baseline_test.go, pprof_har runner.go}; test-benchmark:157
11	benchmarking	same as 10 + PGO pgo-refresh:90 + HelixQA banks/benchmarking-ba
12	ui	THIN local → helix_code/applications/{terminal-ui, desktop} t ui_terminal_interaction_challenge.sh

#	Test type	Exists-where today (helix_code-local + delegated)
13	ux	ABSENT local → only HelixQA challenges/scripts/ux_end_to_end_t
14	Challenges	submodules/challenges/{banks,challenges,cmd} (p1-f06..p2-e2e/challenges/
15	HelixQA-banks / autonomous-QA-session	delegated → HelixQA cmd/helixqa autonomous + pkg/{autonomous,i banks/helixcode-*.yaml; ledger test-coverage.md:66
(mut)	§1.1 paired-mutation meta	helix_code/tests/stresschaos/stresschaos_meta_test.go:80 meta:121
(regr)	regression-guard	helix_code/tests/regression/{critical_paths_test.go, serv

Coverage gaps found: 6 — GAP-1 ddos-local, GAP-2 scaling-local, GAP-3 ui-thin (+pixel-oracle), GAP-4 ux-local, GAP-5 HelixQA-stale/no-sync-channel, GAP-6 regression-guard-registry/D-1..D-7. (ddos/scaling/ux/ui = 4 local-harness gaps per the matrix; +HelixQA-stale +regression-registry = 6 total.)

3. Phase A — close ddos/scaling/ux/thin-ui local gaps

Principle (no delegation-bluff): per CONST-050(B) a delegated HelixQA/challenge slot is acceptable ONLY if it truly exercises HelixCode and captures evidence. Phase A gives EACH of ddos/scaling/ux/ui a **real HelixCode-local harness** AND a CM- *- EVIDENCE gate proving the delegated asset hit a live HelixCode endpoint — not a config-only PASS. Harnesses follow the

`stresschaos.go/stresschaos_meta_test.go` pattern: Go harness writing captured artefacts under `qa-results/<run-id>/`, plus a paired §1.1 meta-test that plants a defect and asserts detection. **§11.4.81 cross-platform:** each harness branches case "`$(uname -s)`" (Linux primary, Darwin adjacent-equivalent or honest SKIP-with-reason).

Task A1 — ddos harness (GAP-1) · owning: SP1/SP2

- **New:** `helix_code/tests/ddos/ddos_harness.go` (sustained request flood vs a real booted HelixCode HTTP server using `httptest/real internal/server`, concurrent flood $N \geq 100$, capture `p50/p95/p99 + rate-limit-rejection` ratio to `qa-results/<run-id>/ddos_latency.json`) + `ddos_meta_test.go` (plant a server with NO rate-limit → assert harness FLAGS unbounded acceptance; plant a 100%-429 server → assert harness records the rejection ratio). Makefile target `test-ddos`.
- **RED:** write `ddos_meta_test.go` first; assert it FAILS against a stub harness that always returns “pass” (proves the harness isn’t a bluff).
- **impl:** real harness driving `internal/server` rate-limit middleware.
- **GREEN+evidence:** make `test-ddos` stdout with real `p50/p95/p99` numbers + `ddos_latency.json` non-empty; meta-test detects planted defects.
- **rollback:** `git checkout -- helix_code/tests/ddos helix_code/` Makefile (new files removable; no prod code touched).

Task A2 — scaling harness (GAP-2) · owning: SP5

- **New:** `helix_code/tests/scaling/scaling_harness.go` exercising `helix_code/internal/worker` SSH worker-pool horizontal scale-out (spawn N in-process workers, measure throughput vs $N=1,2,4,8$; assert near-linear within tolerance; capture `qa-results/<run-id>/scaling_throughput.json`) + `scaling_meta_test.go` (plant a pool that ignores added workers → assert harness detects flat throughput). Makefile `test-scaling`.
- **RED→impl→GREEN+evidence→rollback** as A1; GREEN = throughput table showing scale-out + meta detects the flat-throughput plant.

Task A3 — ux journey harness (GAP-4) · owning: SP4/SP6

- **New:** `helix_code/tests/ux/ux_journey_test.go` driving a full end-user CLI journey programmatically (no human-in-loop, §11.4.98): `init` → `list-models` → `generate` → `command-exec` → assert each step’s real output; capture transcript to `docs/qa/<run-id>/` (§11.4.83 bidirectional transcript). Meta: plant a step that prints a canned string → assert journey asserts on REAL output and FAILS the canned plant.
- **GREEN+evidence:** captured transcript dir + per-step real-output assertions.

Task A4 — ui pixel-oracle harness (GAP-3) · owning: SP4/SP6

- **New:** `helix_code/tests/ui/tui_interaction_test.go` driving the `tview/tcell` TUI via `tcell.SimulationScreen` (inject keys, assert rendered cells) for introspectable UI; for non-introspectable surfaces add a §11.4.117 CV/OCR pixel-oracle fallback (self-validated golden-good/golden-bad fixture pair per §11.4.107(10) — analyzer that PASSes golden-bad → gate FAILs). Honest boundary: where neither hierarchy nor pixel oracle works → SKIP-with-reason (§11.4.3) + tracked operator-attended item (§11.4.52), never fake-PASS.
- **GREEN+evidence:** rendered-cell assertions + OCR analyzer self-validation passing golden-good / failing golden-bad.

Task A5 — delegated-asset evidence gates (anti-delegation-bluff)

- **New:** pre-build gates CM-DDOS-HITS-HELIXCODE, CM-SCALING-HITS-HELIXCODE, CM-UX-HITS-HELIXCODE, CM-UI-HITS-HELIXCODE that assert the corresponding HelixQA challenges/scripts/*.sh emitted captured evidence referencing a live HelixCode endpoint/binary in its last run (parse qa-results/), not a config-only PASS. Each gate gets a paired §1.1 mutation (strip the endpoint-hit assertion → gate FAILs). These convert the “delegated” matrix rows from coverage-bluff risk into evidence-backed delegation.

Catalogue-check (§11.4.74) for Phase A: before writing each harness, survey vasic-digital/HelixDevelopment for an existing load/chaos harness submodule (the stresschaos package is the in-repo precedent to extend, not duplicate). Record Catalogue-Check: extend|no-match in each tracker row.

4. Phase B — HelixQA fetch/pull-to-latest + pointer bump (GAP-5)

Per §11.4.71 (fetch→investigate→integrate) + §11.4.113 (merge-onto-latest-main, **no force-push**) + CONST-055 (post-pull re-verify). Current pin: submodules/helix_qa @ v4.0.0-393-g4d2dcb2 (round 219 / 2026-05-19, STALE).

Task B1 — fetch + investigate

- `git -C submodules/helix_qa fetch --all --prune --tags` (capture stdout).
- `git -C submodules/helix_qa log --oneline HEAD..@{u}` — read EVERY foreign commit body; document what/why/how-it-affects-HelixCode (§11.4.71 step 3). **Do NOT auto-merge blind.**

Task B2 — integrate to latest on a branch, merge-onto-latest-main

- In the submodule: base on latest main tip, merge local (if any) per §11.4.113 6-step; resolve conflicts by judgment (never auto --ours/--theirs); fast-forward push to all upstreams (no force).
- Bump the meta-repo pointer: `git -C <root> add submodules/helix_qa` in the SAME commit as any dependent doc/ledger update (§11.4.71 step + CONST-044 CONTINUATION sync).

Task B3 — CONST-055 post-pull re-verify

- Re-run HelixQA’s own ledger round: confirm submodules/helix_qa/docs/test-coverage.md still shows all 15 slots FILLED post-pull; if any slot regressed to NOT_YET_FILLED, open a tracked issue (§11.4.15 Reopened/Bug) before relying on it.
- Re-run helixqa-build:694 (build HelixQA + DocProcessor/LLMOrchestrator/LLMProvider/VisionEngine) and helixqa-test:707 (-race) to confirm the new pin compiles + green.
- **RED here = the pre-bump state:** capture round-219 staleness as the defect; **GREEN = round updated, ledger re-verified, build+test green, pointer bumped.** Evidence: fetch

`stdout + log HEAD..@{u} + helixqa-test output. Rollback: git -C <root>
checkout -- submodules/helix_qa .gitmodules (revert pointer).`

Operator decision flagged (D-B1): B2 force-push is FORBIDDEN (§11.4.113) — but if the submodule’s mirrors have diverged, the merge-onto-latest-main may surface conflicts needing operator judgment on conflicting cross-session edits. Surface via §11.4.66 AskUserQuestion before merging.

5. Phase C — autonomous-QA-session wiring (§11.4.98 + §11.4.116)

Goal: fully-automated (§11.4.98 — zero human action after startup) autonomous QA sessions executing every registered bank with captured wire evidence, exposed over a real-time §11.4.116 sync-channel the conductor tails live.

Task C1 — §11.4.98 full-automation audit of existing banks

- Classify every HelixCode-facing bank (`submodules/helix_qa/banks/helixcode-*.yaml`) COMPLIANT vs NON-COMPLIANT: a bank requiring a human action mid-run (typing, clicking, hand-triggered webhook) is a §11.4.98 PASS-bluff. Drive programmatically (second account / webhook fixture / loopback). Re-runability proof: PASS at `-count=3` consecutive automated invocations with self-cleaning state.
- **RED:** add a meta-assertion that fails if any in-scope bank contains a manual-step marker. **GREEN:** all banks self-driving; evidence = 3× consecutive clean runs.

Task C2 — §11.4.116 sync-channel on the HelixCode[?] HelixQA bridge

- **Extend** `helix_code/internal/helixqa/wrapper.go` to emit a structured append-only JSONL event stream (one event/line, never rewritten): `session-start / phase-transition / per-bank-start / captured-evidence-path / external-call (LLM/vision/sink-probe) / error / per-bank-verdict` — verdict vocab PASS/FAIL/SKIP/OPERATOR-BLOCKED — PLUS an atomically-rewritten status snapshot (write-temp-then-rename) carrying current session/phase/bank + counters + last verdict.
- **Anti-bluff invariant (§11.4.116):** a verdict event MUST carry the evidence path backing it; a PASS event with no evidence path = channel-layer bluff; a snapshot reporting PASS while the stream shows no evidence event for that bank = contradiction → treat as FAIL.
- **RED:** `wrapper_syncchannel_test.go` asserting (a) a PASS verdict without an evidence-path event FAILS validation; (b) a torn status write is never observed (rename atomicity). **impl:** JSONL emitter + atomic snapshot. **GREEN+evidence:** captured `qa-results/<run-id>/events.jsonl + status.json` with backing evidence paths; meta detects the planted no-evidence PASS. **rollback:** revert `wrapper.go` diff.

Task C3 — autonomous-session runner wired to sync-channel

- New Makefile `test-autonomous-qa`: boots infra (`test-infra-up:182`), runs `cmd/helixqa autonomous` against the live HelixCode stack with `--banks banks/ --validate --tickets`, tailing the C2 sync-channel; the conductor (§11.4.103) tails it live and can §11.4.4-interrupt on a fresh defect.
- **GREEN+evidence:** recorded session timeline + LLM-discovered tickets + per-bank verdicts each with evidence path; re-runnable `-count=3`.

Catalogue-check: the sync-channel format SHOULD reuse the §11.4.116 reference shape if a `vasic-digital/HelixDevelopment` framework already defines it (record `Catalogue-Check`: in the tracker row); else it is new `HelixCode`-side wiring on the existing bridge (no new submodule).

6. Phase D — independent code-review-agent gate per closure (§11.4.125/§11.4.134)

Every SP1/SP2/SP4/SP5/SP6 batch closure routes through a **structurally-separated** code-review agent (subagent-driven §11.4.70) BEFORE the pre-build sweep + any build/tag.

Task D1 — code-review gate marker + script

- **New:** `helix_code/scripts/testing/code_review_gate.sh` producing a fresh `qa-results/code_review/<batch-id>.ok` marker ONLY after a code-review agent analysed: (1) the batch diff + stated intent; (2) captured evidence (§11.4.5/§11.4.69/§11.4.107) + the §11.4.108 runtime-signature registry; (3) blast radius (§11.4.92); (4) git history (reproduces a known-broken pattern? §11.4.114/§11.4.124); (5) quality+safety+will-it-REALLY-work; (6) **every covering test genuinely catches its negation** — a test that PASSES on broken-for-user work, or a gate whose paired §1.1 mutation does NOT make it FAIL, is a finding.
- **New gate:** `CM-CODE-REVIEW-GATE-BEFORE-BUILD` — pre-build sweep + main build refuse to start without a fresh marker for the current batch (produced after the LAST fix). Paired §1.1 mutation: backdate/remove the marker → gate FAILs.

Task D2 — §11.4.134 iterate-until-GO loop

- On ANY finding (blocking, nit, OR warning) the batch is fixed/improved/test-covered (four-layer §11.4.4(b), RED-first §11.4.43/§11.4.115) and the review is **RE-RUN**; loop terminates ONLY on a clean GO with ZERO findings AND ZERO warnings. Every round's verdict carries rock-solid PHYSICAL captured evidence (§11.4.5/§11.4.69/§11.4.107); a GO unbacked by captured evidence is itself a §11.4 review-loop bluff.
 - **RED:** meta-test plants a residual warning → asserts the loop does NOT emit the `.ok` marker. **GREEN:** marker emitted only on zero-finding GO. **rollback:** remove script + gate (no prod code).
 - Use the repo's `code-review/superpowers:requesting-code-review` skill as the reviewing agent; the review conclusions are captured under `qa-results/code_review/<batch-id>/.`
-

7. Phase E — regression-guard registration per fixed defect (incl. D-1..D-7) §11.4.135

Goal: a STANDING regression-guard suite that runs on EVERY build+deploy, runs FIRST (highest-risk set, §11.4.132), and BLOCKS the tag on any failure. Every closed defect registers a permanent §11.4.115 RED-on-broken-artifact guard in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step). Closure without a guard = §11.4.123 violation.

Task E1 — guard registry scaffold (GAP-6)

- **New:** `helix_code/tests/regression/guards/registry + guard_registry_test.go` runner; each guard is `<DEFECT-ID>_guard_test.go` with a single `RED_MODE` **polarity switch** (env, default 1 = reproduce-the-defect-on-pre-fix-artifact + assert present; 0 = standing GREEN guard asserting defect ABSENT). One source, two roles (§11.4.115). Makefile `test-regression-guards` (runs `RED_MODE=0` set; gated into `test-full/release`).
- **New gates:** `CM-REGRESSION-GUARD-REGISTERED` (every closed defect id has a guard file) + `CM-REGRESSION-GUARD-SUITE-WIRED` (suite runs in release gate) + paired §1.1 mutation (delete a guard → gate FAILs).

Task E2 — D-1..D-7 guards (each = RED→impl→GREEN+evidence→rollback)

Each guard's RED captures the historical defect on the pre-fix artifact; GREEN asserts absence on the fixed artifact (clean target, §11.4.108/§11.4.139). These are authored alongside the owning SP's fix but the GUARD rows are SP7's deliverable.

Guard	Defect
<code>D1_completion_models_guard_test.go</code>	D-1 hardcoded 3-model list <code>helix_agent/</code> <code>completion.go:406</code>
<code>D2_cli_listmodels_guard_test.go</code>	D-2 <code>cmd/cli/main.go:1361</code> lists failed/p
<code>D3_loadapikeys_wired_guard_test.go</code>	D-3 dead secrets. <code>LoadAPIKeys</code> intern <code>loader.go:30</code>
<code>D4_working_model_filter_guard_test.go</code>	D-4 filter loaded-never-applied <code>adapter.go</code>
<code>D5_provider_hardcode_guard_test.go</code>	D-5 hardcoded lists <code>openai_provider.go:203</code> ,anthropic
<code>D6_cli_agent_exec_guard_test.go</code>	D-6 stub returns hardcoded strings <code>clis/age</code> <code>qwencode.go:101/115/137</code>

Guard	Defect
D7_replace_dev_helix_agent_guard_test.go	D-7 helix_code/go.mod lacks replace

- **Anti-bluff:** each guard's RED MUST genuinely FAIL on the current (pre-fix) artifact (§11.4.115) — a RED that passes on the known-broken artifact is a blind test (finding). D-1/D-6 guards live partly in `helix_agent` (its own regression suite per CONST-047) with a HelixCode integration mirror once D-7's `replace` lands.
- **rollback:** each guard is additive test-only; `git checkout -- helix_code/tests/regression/guards/<file>`.

Task E3 — risk-ordered execution (§11.4.132)

- Wire the guard suite to run FIRST in the post-deploy cycle, ordered by recency × problematic-history × reopen-count; D-1..D-7 (most-recently-worked) run before the broader suite. Only after the guard set is GREEN with captured evidence does the rest of `test-full` run.

8. Operator decisions required

- **D-OP-1 (delegation vs local for ddos/scaling/ux/ui):** Phase A builds REAL HelixCode-local harnesses for all four. Confirm this (vs. accepting evidence-backed HelixQA delegation via the Task A5 gates only). *Recommended: build local harnesses* — removes the CONST-050(B) coverage-bluff risk permanently. (§11.4.66)
 - **D-OP-2 (HelixQA merge conflicts):** if Phase B2's merge-onto-latest-main surfaces conflicting cross-session edits in the submodule, operator judgment is needed (force-push is forbidden §11.4.113). Surface via `AskUserQuestion` before merging.
 - **D-OP-3 (new flow-submodule QA scope, SP6):** each new flow submodule (`flow_engine/dag_orchestrator/pipeline_runtime/agent_mesh`) needs its OWN full 15-type matrix + banks + Challenges (CONST-050B/CONST-051). Confirm SP7 owns authoring those, or each SP6 submodule owns its own QA. *Recommended: each submodule owns its standalone matrix; SP7 owns the cross-cutting autonomous-session + guard registry.*
 - **D-OP-4 (pixel-oracle dependency, Task A4):** §11.4.117 CV/OCR fallback may need an OCR/CV dependency (tesseract/opencv). Confirm install vs. SKIP-with-operator-attended-migration for non-introspectable UI surfaces.
 - **D-OP-5 (autonomous-session compute budget, Phase C):** full autonomous banks at `-count=3` + live infra is compute-heavy; confirm it runs in background (§11.4.89) per cycle vs. release-gate-only.
-

9. Summary

1. SP7 is cross-cutting QA binding SP1/SP2/SP4/SP5/SP6; verified against analysis-F §3 + master-roadmap §SP7 by direct file inspection.
2. HelixCode-local coverage is STRONG for unit/integration/e2e/security/chaos/stress/performance/benchmarking, with a real §1.1 meta-test harness (`stresschaos_meta_test.go:80-193`).
3. **Coverage gaps found: 6** — GAP-1 ddos-local, GAP-2 scaling-local, GAP-3 ui-thin(+pixel-oracle), GAP-4 ux-local, GAP-5 HelixQA-stale/no-sync-channel, GAP-6 regression-guard-registry(+D-1..D-7).
4. ddos/scaling/ux assets exist ONLY as HelixQA challenge scripts — delegated slots that risk a CONST-050(B) coverage-bluff unless evidence-backed.
5. Phase A builds REAL HelixCode-local ddos/scaling/ux/ui harnesses (stresschaos pattern) + CM- *-HITS-HELIXCODE anti-delegation-bluff gates, each with paired §1.1 mutation.
6. Phase B fetches/pulls HelixQA to latest (round 219 → current) via §11.4.71 + §11.4.113 merge-onto-latest-main (no force), bumps the `.gitmodules` pointer, and CONST-055 re-verifies the 15-slot ledger.
7. Phase C audits banks for §11.4.98 full-automation (no human-in-loop, `-count=3` re-runnable) and wires a §11.4.116 JSONL sync-channel + atomic status snapshot onto `internal/helixqa/wrapper.go`, with the evidence-path-backed-verdict invariant.
8. Phase D adds the §11.4.125 code-review gate (CM-CODE-REVIEW-GATE-BEFORE-BUILD) and §11.4.134 iterate-until-GO loop (zero findings AND zero warnings, physical evidence per round).
9. Phase E builds the §11.4.135 standing regression-guard registry with RED_MODE polarity switch (§11.4.115) and authors D-1..D-7 guards, run FIRST risk-ordered (§11.4.132) and tag-blocking.
10. Every task is RED→impl→GREEN+evidence→rollback; new harnesses/guards are additive test-only (clean rollback); no production code is removed (§11.4.122).
11. Catalogue-check (§11.4.74) precedes each new harness; the in-repo stresschaos package is the extend-precedent — no duplicate load/chaos engine.
12. 5 operator decisions flagged (D-OP-1 local-vs-delegated, D-OP-2 HelixQA merge conflicts, D-OP-3 SP6 submodule QA ownership, D-OP-4 pixel-oracle dependency, D-OP-5 autonomous-session compute budget).

Coverage gaps found: 6.